

Ejercicio 8

- ¿Qué hace cada uno de los métodos que aparece a continuación?
- Calcule la función de tiempo de ejecución y el orden de la misma para cada método.
- ¿Cuál de los dos métodos es más eficiente? ¿Por qué?

```
static public int pIter(int x, int n){
    int potencia;
    if (n == 0)
        p = 1;
    else {
        if (n == 1)
            p = x;
        else{
            p = x;
            for (int i = 2 ; i <= n ; i++) {
                p *= x ;
            }
        }
    }
    return p;
}

static public int pRec( int x, int n){
    if( n == 0 )
        return 1;
    else{
        if( n == 1)
            return x;
        else{
            if ( (n % 2 ) == 0)
                return pRec (x * x, n / 2 );
            else
                return pRec (x * x, n / 2) * x;
        }
    }
}
```

}

- a. Los métodos calculan la potencia de orden n de x . La diferencia es que `pIter` lo hace iterativamente mientras que `pRec` lo hace recursivamente.
- b. Calculemos las funciones y el orden de ejecución de ambos métodos.

- pIter

Primero tenemos varias asignaciones que son operaciones constantes y las vamos a representar con la constante c_1 . Después tenemos un `for` que vamos a representar mediante una sumatoria.

Como el primer valor que tomar i es 2, ese va a ser el límite inferior de la sumatoria. Además, el índice aumenta de a uno, así que podemos tomar el límite superior como n .

La sumatoria nos va a quedar,

$$\sum_{i=2}^n c_2$$

Podemos escribir la función $T(n)$ como

$$T(n) = c_1 + \sum_{i=2}^n c_2$$

Ahora vamos a usar trucos matemáticos para hacer volar las variables de iteración. Propiedades a usar:

$$- \sum_{k=m}^n c = c \cdot (n - m + 1)$$

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{i=2}^n c_2 \\ &= c_1 + c_2(n - 2 + 1) \\ &= c_1 - c_2 + c_2n \end{aligned}$$

$$\begin{aligned} \text{Renombrando } c_3 &= c_1 - c_2. \\ &= c_3 + c_2n \end{aligned}$$

Función de tiempo de ejecución:

$$\boxed{T(n) = c_3 + c_2n}$$

Vamos a demostrar que

$$\boxed{T(n) = c_3 + c_2n \leq O(n)}$$

buscando los valores de k y n_0 que acoten la función.

- Primer término c_3 Sabemos que $1 \leq n$. Si multiplicamos a ambos lados por c_3 .

$$c_3 \leq c_3 \cdot n$$

Ya tenemos el valor de $k_1 = c_3$. Si tomamos $n_0 = 1$, la desigualdad se cumple

$$c_3 \leq c_3(1)$$

- Segundo término Sabemos que $n \leq n$. Si multiplicamos a ambos lados por c_2 .

$$c_2 \cdot n \leq c_2 \cdot n$$

Ya tenemos el valor de $k_2 = c_2$. Si tomamos $n_0 = 0$, la desigualdad se cumple

$$c_2 \cdot 0 \leq c_2 \cdot 0$$

Sumamos todos los resultados parciales.

$$c_3 + c_2 n \leq k_1 n + k_2 n$$

$$T(n) \leq (k_1 + k_2)n$$

$$T(n) \leq (c_3 + c_2)n$$

Renombrando $c = c_3 + c_2$

$$T(n) \leq cn$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n), \text{ con } c = c_3 + c_2 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

- pRecur Tenemos un método recursivo con dos casos base que realizan una función constante.
El caso base lo representamos con c_1 para los $n = 0$ y $n = 1$.

$$c_1 \quad n \leq 1$$

Por otro lado, para otros valores de n , se realiza una llamada recursiva.

$$c_2 + T\left(\frac{n}{2}\right) \quad n > 1$$

Podemos escribir la función $T(n)$ como

$$T(n) = \begin{cases} c_1 & n \leq 1, \\ c_2 + T\left(\frac{n}{2}\right) & n > 1. \end{cases}$$

Ahora hay que ver como va cambiando la expresión con las llamadas recursivas para llegar a un paso general.

1. $T(n) = c_2 + T(\frac{n}{2})$
2. $T(n) = c_2 + (c_2 + T(\frac{n}{2 \cdot 2})) = 2c_2 + T(\frac{n}{2^2})$
3. $T(n) = 2c_2 + (c_2 + T(\frac{n}{2 \cdot 2^2})) = 3c_2 + T(\frac{n}{2^3})$
- $\vdots$

Paso i (paso general) $T(n) = i \cdot c_2 + T(\frac{n}{2^i})$ Ahora hay que despejar i de la expresión que quedó "como parámetro" a la llamada recursiva para obtener el caso base.

Como el caso base se ejecuta cuando $n \leq 1$, vamos a tomar el 1. O sea,

$$\begin{aligned}\frac{n}{2^i} &= 1 \\ n &= 2^i \\ i &= \log_2 n\end{aligned}$$

Reemplazamos el caso base en la función que obtuvimos como paso general. Propiedades usadas:

$$- a^{\log_a b} = b$$

$$T(n) = \log_2 n \cdot c_2 + T(\frac{n}{2^{\log_2 n}})$$

$$T(n) = c_2 \cdot \log_2 n + T(1)$$

$$T(n) = c_2 \cdot \log_2 n + c_1$$

Entonces, la función de tiempo de ejecución de pRecur es

$$\boxed{T(n) = c_2 \cdot \log_2 n + c_1}$$

Vamos a demostrar el orden de la función. Hay que demostrar que

$$\boxed{T(n) = c_2 \cdot \log_2 n + c_1 \leq O(\log_2(n))}$$

Hay que buscar los valores constantes c y n_0 tales que

$$c_2 \cdot \log_2 n + c_1 \leq c \cdot \log_2(n), \text{ para todo } n \geq n_0$$

– Primer término, $c_2 \cdot \log_2 n$

$$\log_2 n \leq \log_2 n$$

Multiplicamos ambos términos por c_2 .

$$c_2 \cdot \log_2 n \leq c_2 \cdot \log_2 n$$

Con esto hallamos el valor de k_1 , es $k_1 = c_2$. Esta desigualdad vale para $n_0 = 1$ (el logaritmo no está definido para $n_0 = 0$).

- Segundo término: c_1 Sabemos que $1 \leq \log_2 n$. Si multiplicamos a ambos lados por c_1 .

$$c_1 \leq c_1 \log_2 n$$

Tenemos el valor de la constante $k_2 = c_1$. Y la desigualdad se cumple para $n_0 \geq 1$.

Sumamos todos los miembros los resultados anteriores de cada lado.

$$T(n) = c_2 \cdot \log_2 n + c_1 \leq k_1 \log_2 n + k_2 \log_2 n$$

$$T(n) \leq (k_1 + k_2) \cdot \log_2 n$$

$$T(n) \leq (c_2 + c_1) \cdot \log_2 n$$

$$\text{Renombrando } c = c_2 + c_1$$

$$T(n) \leq c \cdot \log_2 n$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(\log_2 n), \text{ con } c = c_2 + c_1 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1$$

c. El método pIter es más eficiente que pRec porque $O(n) > O(\log_2 n)$.